МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ И РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

А.А. Романов

КОНСТРУИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Лабораторный практикум

Ульяновск
УлГТУ
2022

УДК 004.8(076)
ББК 32.813.5я7
Р 69

Рекомендовано научно-методической комиссией факультета информационных систем и технологий в качестве практикума.

Рецензент начальник отдела развития и поддержания интегрированной автоматизированной системы управления предприятием ФНПЦ «НПО «Марс» АО «НПО «МАРС» А. А. Перцев

Романов, Антон Алексеевич

Р 69 КОНСТРУИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ : лабораторный практикум / А. А. Романов. – Ульяновск : УлГТУ, 2022. – 34 с.

Практикум адресован студентам для выполнения лабораторных работ и курсового проекта по дисциплине «Конструирование программного обеспечения». Рекомендации и требования к курсовому проекту разработаны в соответствии с рабочей программой дисциплины. Предназначены для студентов, обучающихся по укрупненной группе специальностей и направлений 09.00.00 «Информатика и вычислительная техника».

Работа подготовлена на кафедре «Информационные системы».

УДК 004.8(076)
ББК 32.813.5я7

© Романов А.А., 2022
© Оформление. УлГТУ, 2022

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 4
ЛАБОРАТОРНАЯ РАБОТА №1..... 5
ЛАБОРАТОРНАЯ РАБОТА №2..... 8
ЛАБОРАТОРНАЯ РАБОТА №3..... 12
ЛАБОРАТОРНАЯ РАБОТА №4..... 14
ЛАБОРАТОРНАЯ РАБОТА №5..... 15
ЛАБОРАТОРНАЯ РАБОТА № 6..... 17
ЛАБОРАТОРНАЯ РАБОТА №7..... 18
КУРСОВОЙ ПРОЕКТ..... 29
ПЕРЕЧЕНЬ КОНТРОЛЬНЫХ ВОПРОСОВ К ЭКЗАМЕНУ..... 32
СПИСОК ЛИТЕРАТУРЫ..... 34

ВВЕДЕНИЕ

Целью освоения дисциплины «Конструирование программного обеспечения» является изучение методов и технологий разработки программного обеспечения, а также программирования, конструирования и тестирования программных продуктов.

Задачами дисциплины являются:

- изучение методологий разработки программных продуктов;
- формирование навыков аналитики предметной области, декомпозиции задач, оценки объемов и планирования работ;
- разработка и тестирование программной системы по изученным методологиям.

Кроме того, в результате изучения дисциплины «Конструирование программного обеспечения» обучающиеся на основе приобретенных знаний, умений и навыков достигают освоения компетенций на определенном уровне их формирования.

Учебным планом направления 09.03.04 «Программная инженерия» предусмотрен курсовой проект.

ЛАБОРАТОРНАЯ РАБОТА №1

Тема: Анализ предметной области. Декомпозиция задач.

Общие сведения по работе

Анализ предметной области при конструировании программного обеспечения необходим для достижения нескольких целей:

1. Уточнения общего задания на разработку. Формализация задания даже после создания технического задания может быть недостаточной. Причинами могут являться: недостаточная проработка технического задания, быстро изменяющиеся требования, которые не всегда можно своевременно отразить в документации, высокая зависимость от технологий конструирования, что приводит к неопределенностям в реализации, т. д.
2. Планирование процессов конструирования на основе существующего задания. Это своеобразный «мост» между требованиями технического задания и заданиями для менеджеров, разработчиков, тестировщиков и других специалистов.
3. Выбор и оценка технологий реализации проекта. Поставленные задачи могут иметь неопределенность в части выбора технологий и инструментов реализации. В таком случае от специалистов команды разработки требуется провести анализ, оценить альтернативы и критически обосновать тот или иной выбор.

Процесс анализа может быть многоступенчатым, основанном на взаимодействии различных участников команды разработки. Основополагающим принципом будет являться анализ исходной информации и декомпозиция на более конкретно сформулированные задачи. Для всесторонней оценки полноты описания задачи нужно как

можно точнее описать результат и критерии оценки готовности задачи. Излагать задачу, результат и критерии необходимо в структурированном виде, пользуясь общепринятыми терминами. Под результатом подразумевается описание созданных частей программной системы: модулей, подпрограмм, ресурсов, позволяющих выполнять требуемые функции. Критерии готовности позволяют уточнить понимание, когда результат считается достигнутым. Критерии также позволяют упростить процессы конструирования:

1. Позволяют согласовать понимание задач между заказчиком и командой разработки. Этот пункт может показаться противоречивым, ведь процесс конструирования уже был начал по требованиям заказчика. Но важной особенностью здесь будет являться изменчивость самих требований и внешних по отношению к проекту условий. Таким образом подход с уточнением требований является важным в современных условиях разработки.
2. Выступают источником ожидаемого поведения при проектировании тестов и проведении тестирования.
3. Позволяют создавать документацию на основе описанных задач, например, использоваться в приемочном испытании.

Задания к лабораторной работе

1. По теме курсовой работы, исходя из требований к функциям программной системы, выделить перечень задач. На данном этапе задачи формулируются без каких-либо технических подробностей.
2. Для каждой задачи из списка описать результат и критерии оценки его готовности. Критерии должны быть понятными и однозначно свидетельствовать о завершении или не завершении работы над задачей.

3. Если результат по задаче трудно представим, то либо попытаться разбить задачу на более мелкие части, либо переформулировать задачу (в том числе и поставить задачу на исследование, если существует некоторая неопределенность в результате).

Контрольные вопросы

1. Что может служить исходной информацией для определения задач по проекту?
2. От чего зависит количество задач?
3. Как зависят риски проекта от размера выделенных задач?
4. Как связана производительность команды и выделенные задачи?
5. Какую дополнительную информацию можно привлекать для уточнения списка задач?

ЛАБОРАТОРНАЯ РАБОТА №2

Тема: Оценка объема работ по задачам.

Общие сведения по работе

Оценка – прогнозное значение, выражающееся во времени, ресурсах для реализации проекта. При этом оценка не всегда связана с датами проведения работ. Хорошая оценка работа по проекту позволяет не только строить календарные графики, но и получать актуальную информацию о ходе проекта, позволяет управлять им. Ошибка в оценке проекта может складываться из неточной информации об оцениваемом программном продукте и неточности, вызванной самим процессом оценки. Первую часть проблем нужно устранять подробным анализом требований, общением с заказчиком (см. лабораторную работу №1). Вторая часть проблем решается при помощи методов оценки.

Методы оценки:

1. Оценка по аналогии. Такой подход основывается на ранее полученном опыте конструирования программных продуктов. Данный метод позволяет, используя накопленную статистику по выполненным задачам, произвести оценку новых, в том случае, если требования к ним близки. Конечно, каждый проект по своему уникален, отличается набором функций, есть разница в квалификации специалистов, создающих его, проекты различаются набором технологий и прочими условиями реализации. Но статистические данные тем и хороши, что с ростом объема анализируемых данных повышается точность прогнозов. Ранее считавшиеся отклонениями от нормы оценки по определенному виду работ с ростом числа таких ситуаций становятся закономерностью.

2. Экспертная оценка. Экспертная оценка строится специалистом, знакомым с проблемной областью, технологиями разработки, требованиями к функциям конструируемого приложения. Последовательность операций при экспертной оценке включает подсчет всех создаваемых сущностей, ресурсов, веб-страниц, отчетов, диалогов, функций, сценариев использования. Такой подход позволяет, во-первых, в количественном масштабе оценить сложность проекта, во-вторых, повторно обратиться к рассмотрению и анализу проекта и решить, есть ли ответ на вопрос «Все ли известно о текущем проекте в мельчайших подробностях?». После подсчета необходимо оценить каждую из составных частей. Прогнозная оценка в таком случае будет тем точнее, чем больше подробностей имеется о проекте.
3. Взвешенные оценки. Эта группа методов подразумевает, что предположение только одного значения оценки обладает неточностью. Более точный вариант – получение ряда значений, например, от нескольких участников проекта, и усреднение. Еще один способ – PERT, оценка по трем точкам, предполагает оценку трудоемкости в случае оптимистического развития событий, когда риски не реализуются, пессимистического и некоторого среднего значения.

В каких величинах лучше всего отразить трудоемкость? Обычно мерой трудоемкости являются временные величины, приходящиеся на одного гипотетического исполнителя: человеко-часы, человеко-дни, … При формулировании трудоемкости решение обычно принимается в пользу такой меры, которая позволит эффективно управлять проектом, отслеживать изменения в нем и отражать состояние. Так если масштаб проекта выразить в человеко-месяцах, то сама мелкая целая единица будет

1 месяц, т. е. частота контроля и принятия решений будет соответствовать одному месяцу. При использовании современных методологий разработки это неприемлемо. Поэтому чаще всего оценивают трудоемкость в человека-часах.

По своему виду оценка может быть также выражена точечной оценкой (6 часов) или интервальной (4-8 часов). Выбор представления зависит от степени определенности задачи. Точечные оценки нужны там, где задачи описаны детально, где нет рисков увеличения сроков работ. Интервальные оценки полезны для описания трудоемкости задач, в которых есть неопределенность в силу ряда факторов, и она может в разных задачах быть различной. Поэтому ширина диапазона в первую очередь отражает степень уверенности в полноте описания задачи. В качестве примеров задач см. таблицу 1.

Таблица 1. Примеры вариантов оценок задач

Задача	Точечная оценка	Диапазонная оценка
#1	1.5	1.25 – 2.0
#2	1.5	1.5 – 2.5
#3	2.0	2.0 – 3.0

Из примеров видно, что точечные оценки задач #1 и #2 совпадают, в то время как интервальные могут отличаться и несут в себе дополнительную информацию. Подробнее см. в [3].

Задания к лабораторной работе

1. Взять список задач из первой лабораторной.
2. Проанализировав целевой результат каждой задачи, произвести ее оценку в часах, исходя из собственного опыта. Рекомендуется использовать интервальную оценку. Подсчитать итоговую сумму по всем задачам.

3. Переоценить задачи, если они получаются слишком объемными (для курсового проекта, как правило, оценка одной задачи не превышает 6-8 часов), либо разбить их на более мелкие.
4. Соотнести полученные оценки с прочими активностями в семестре и оценить выполнимость проекта в запланированном объеме.
5. При работе над курсовым проектом фиксировать фактическое время работы над задачей.

Контрольные вопросы

1. Дайте определение цели оценки объема работ
2. Каковы риски при переоценке или недооценке работ?
3. Каковы факторы, влияющие на величину ошибки при оценке задач?
4. Какие существуют методы оценки?
5. Какова должна быть детализация при оценке задач?

ЛАБОРАТОРНАЯ РАБОТА №3

Тема: Планирование разработки. Приоритезация задач.

Общие сведения по работе

Конструирование программного обеспечения является сложным процессом, включающим как различные виды работ (анализ, проектирование, кодирование, тестирование, …), так и распределяющим объемы задач по этим видам работ. Во внимание нужно привлекать: вид жизненного цикла разработки, оценку задач, требования к реализации, технологические аспекты, квалификацию разработчиков. Подробнее см. в [3].

Задания к лабораторной работе

1. Взять список задач и их оценки из второй лабораторной работы.
2. Выбрать одну из моделей жизненного цикла разработки и в рамках нее выполнить пункты 3-5. Здесь требуется приложить описание применения методологии, инструментов, технологий, работы в группе (если над проектом работает больше одного человека).
3. Пересортировать список задач по приоритетности их выполнения. Здесь уже стоит учитывать технические детали реализации.
4. Выделить этапность для частичной сдачи проекта: архитектура, прототип, этапы реализации (может быть несколько), тестирование, доработка, если потребуется. На этапы дополнительно выделить время, добавив к исходной оценке.
5. Сформировать календарный график работ в семестре над проектом, т. е. выполнить привязку к датам.

Контрольные вопросы

1. В чем состоит процесс управления разработкой?
2. Каким образом происходит разбиение работ на этапы?
3. Какими способами можно оценить объем работ на каждый этап?
4. Как измеряется скорость работы команды?
5. В чем разница планирования проектов, разрабатываемых по гибким и тяжеловесным методологиям?

ЛАБОРАТОРНАЯ РАБОТА №4

Тема: Проектирование в конструировании.

Общие сведения по работе

В ходе освоения дисциплины студенты должны проявить аналитический склад ума и объединить познания из нескольких дисциплин. Данная лабораторная работа позволяет студентам проявить навыки проектирования программных систем в сочетании с приобретенными навыками по конструированию программного обеспечения. Разные методологии разработки определяют разный объем проектирования. Анализ должен вывить необходимые объемы проектирования, а студент должен уметь аргументировать свое решение. Проектирование необходимо выполнить для достижения целей конструирования: сокращения трудоемкости, минимизации сложности, повышения управляемости проектом, повышения качества процессов и продукта. Подробнее см. в [3].

Задания к лабораторной работе

1. По ранее разработанным требованиям и списку выделенных задач сформировать проект.
2. Содержание проекта выбрать самостоятельно.

Контрольные вопросы

1. Как происходит процесс проектирования в гибких и тяжеловесных методологиях?
2. Каков объем документации в гибких и тяжеловесных методологиях?
3. Как распределяются роли и ответственность при проектировании?
4. Как происходит проектирование в условиях неопределенности?
5. Какие системы используются для ведения документации?

ЛАБОРАТОРНАЯ РАБОТА №5

Тема: Разработка прототипа программной системы.

Общие сведения по работе

Основная цель прототипирования — снижение неопределенности в требованиях к конструируемому программному продукту. Это процесс создания модели требуемого программного продукта. Модель может принимать одну из трех форм:

1. Бумажный макет пользовательского интерфейса или прототип, созданный при помощи CASE-средств (изображает человеко-машинный диалог).
2. Работающий прототип. Чаще всего это программная система, выполняющая некоторую часть требуемых функций.
3. Существующая программа, характеристики которой затем должны быть улучшены.

Последовательность действий при прототипировании:

1. Анализ требований к разрабатываемой программной системе.
2. Выполнение проектирования в минимальном объеме. Сосредотачиваются на тех характеристиках программной системы, которые решают пользовательские задачи.
3. Создание прототипа на основе минимального проекта.
4. Прототип оценивается заказчиком. На его основе происходит уточнение требований.

Указанные шаги повторяются до тех пор, пока не будут выявлены все требования.

Задания к лабораторной работе

1. Выделить из списка задач несколько для формирования прототипа.
2. Реализовать. Реализация задач может быть неполной.

3. Сделать выводы по планам проекта, т. к. целью создания прототипа является как уточнение задач по проекту с преподавателем, так и оценка собственных возможностей по работе над проектом с учетом используемых технологий, оценок задач.

Контрольные вопросы

1. Каковы цели разработки прототипа программной системы?
2. Какие есть ограничения у прототипа?
3. Каковы риски для проекта при прототипировании?
4. Какие роли включены в процесс прототипирования?
5. Каков жизненный цикл прототипа?

ЛАБОРАТОРНАЯ РАБОТА № 6

Тема: Итоги работы, отчет.

Общие сведения по работе

В работе необходимо отразить состояние по ранее поставленным задачам: статус, фактически затраченное время. Данная процедура необходима для формирования понимания сути и сложностей в выполненной работе, получение опыта при работе над новыми задачами. Кроме того, необходимо описать результат разработки программной системы с характеристикой основных процессов, инструментов и технологий разработки.

Задания к лабораторной работе

1. По результатам реализации сформировать отчет о выполнении задач, фактического времени их выполнения, появления новых задач.
2. Также приложить описание результатов реализации программной системы.

Контрольные вопросы

1. Как ведется разработка при наличии плана работ?
2. Как происходит управление проектом при разработке?
3. Какие показатели влияют на управление разработкой проекта?
4. Как измеряется скорость работы команды?
5. Как можно адаптировать команду под условия разработки?

ЛАБОРАТОРНАЯ РАБОТА №7

Тема: Тестирование в конструировании.

Общие сведения по работе

Классификация видов тестирования отражает многообразие походов организации контроля качества программных систем. Понимание классификации позволяет определить возможные варианты процедур тестирования, оценить трудоемкость, спланировать тестирование, выбрать подходящие инструменты. Это знание является необходимым при построении жизненного цикла разработки программных систем. Навыки использования видов тестирования для контроля функционирования программных систем позволяют тестировщику быстрее локализовать дефекты, создавать более полные наборы тестов.

Деление на различные виды тестирования позволяет применять только подходы соразмерно требованиям к проверке поставленных требований, т. е. отдельные виды тестирования могут получить повышенный приоритет выполнения, в то время как все прочие виды тестирования будут не так важны. Выбор видов тестирования зависит от поставленных требований (зачастую в техническом задании уже сформулированы требования приемочного тестирования, которые нужно будет проверить при готовности системы), этапа разработки, условий использования системы. Используемые виды тестирования можно использовать в чистом виде (например, тестирование пользовательского интерфейса), так и охарактеризовать с позиций типов, методов и других видов тестирования (ручное функциональное тестирование, автоматизированное функциональное тестирование, регрессионное ручное функциональное тестирование, и т. д.)

Представление классификации основано на делении следующим ключевым группам:

1. Уровни тестирования предполагают оценивать программную систему с точки зрения объема программных модулей и учета соответствующих аспектов (например, внимание будет сосредоточено только на проверке кода, реализующего бизнес-логику, или важным будет являться момент интеграции подсистем или модулей). В зависимости от степени детальности компонентов уровни подразделяются на:

– Компонентное (модульное) – тестирование программы на уровне отдельно взятых модулей, функций или классов. Цель компонентного тестирования – выявление локализованных в модуле ошибок в реализации алгоритмов, а также определение степени готовности системы к переходу на следующий уровень разработки и тестирования.

– Интеграционное тестирование предназначено для проверки связи между компонентами, а также взаимодействия с различными частями системы (операционной системой, оборудованием либо связи между различными системами). Уровнями интеграционного тестирования являются: компонентный интеграционный (проверяется взаимодействие между компонентами системы после проведения компонентного тестирования), системный интеграционный уровень (проверяется взаимодействие между разными системами после проведения системного тестирования).

– Системное тестирование проверяет как функциональные, так и нефункциональные требования к системе в целом. При этом выявляются дефекты, такие как неверное использование ресурсов системы, не предусмотренные комбинации данных пользовательского уровня, несовместимость с окружением, непредусмотренные сценарии

использования, отсутствующая или неверная функциональность, неудобство использования и т. д.

При этом деление на уровни тестирования предполагает участие различных ролей участников команды разработки. Предполагается, что тесты уровня компонентного тестирования разрабатывает сам программист, контролируя самостоятельно качество своей работы или работы своих коллег. Интеграция модулей или общесистемное тестирование могут быть проведены уже другими участниками команды, знающими требования к программным интерфейсам модулей и подсистемам и имеющими возможность для манипуляции с ними.

2. Типы тестирования сосредоточены на делении видов тестирования, проверяющих наличие требуемой функциональности (функциональный тип тестирования, что система делает) и проверяющих режимы работы системы при выполнении пользователями своих задач (нефункциональный тип тестирования, как система это делает).

Функциональный тип тестирования включает следующие подвиды:

– функциональное тестирование;

– тестирование возможности взаимодействия (включает тестирование совместимости и интеграционное тестирование);

– тестирование безопасности.

Нефункциональный тип тестирования описывает такие подвиды, как:

– нагрузочное тестирование;

– стресс-тестирование;

– тестирование удобства пользования;

– тестирование надежности и т. д.

Примеры проведения нефункциональных тестов:

– общее исследование поведения системы при экстремальных нагрузках;

– экспертиза обработки ошибок и исключений;

– тестирование пропускной способности системы;

– изучение некоторых частей системы или ее компонентов в условиях непропорциональной нагрузки.

3. Методы тестирования описывают применение видов тестирования в зависимости от объекта тестирования. Методы подразделяются на статические (тестирующие такие объекты, как программный код, документацию, ресурсы проекта, все то, что может быть проверено без непосредственного выполнения программного кода, т. е. запуска программной системы) и динамические (тестирующие функциональность программной системы и режимы ее эксплуатации).

Подвидом статического метода тестирования является рецензирование – вид тестирования, который может проводиться перед динамическим тестированием.

Как он может быть проведен:

– вручную,

– с применением анализаторов кода.

Рецензирование может проводится для любого продукта, связанного с разработкой программного обеспечения, включая спецификации требований и дизайна, код, планы тестирования, руководства пользователя и т. п. Во время рецензирования могут быть найдены упущения, например в требованиях, которые маловероятно найти во время динамического тестирования. Более широким видом статического тестирования является статический анализ, проводимый при помощи специальных инструментов. При этом анализируется:

– код программы (например, потоки управления и поток данных),

– сгенерированный код, например HTML, XML.

На рисунке 1 приведен пример скриншота отчета статического анализатора кода SonarQube с результатами анализа, охарактеризованных по степени серьезности, с рекомендацией к устранению. На рисунке 2 проводится отчет о валидации html разметки сайта ulstu.ru.

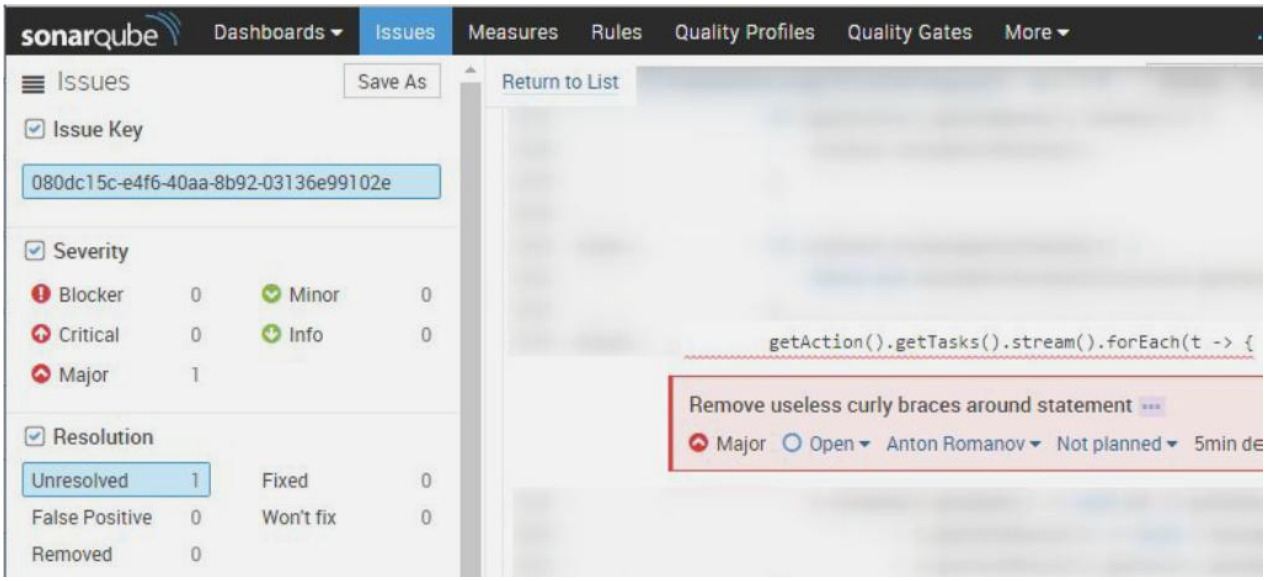


Рис. 1. Отчет в системе SonarQube о статическом анализе программного кода проекта

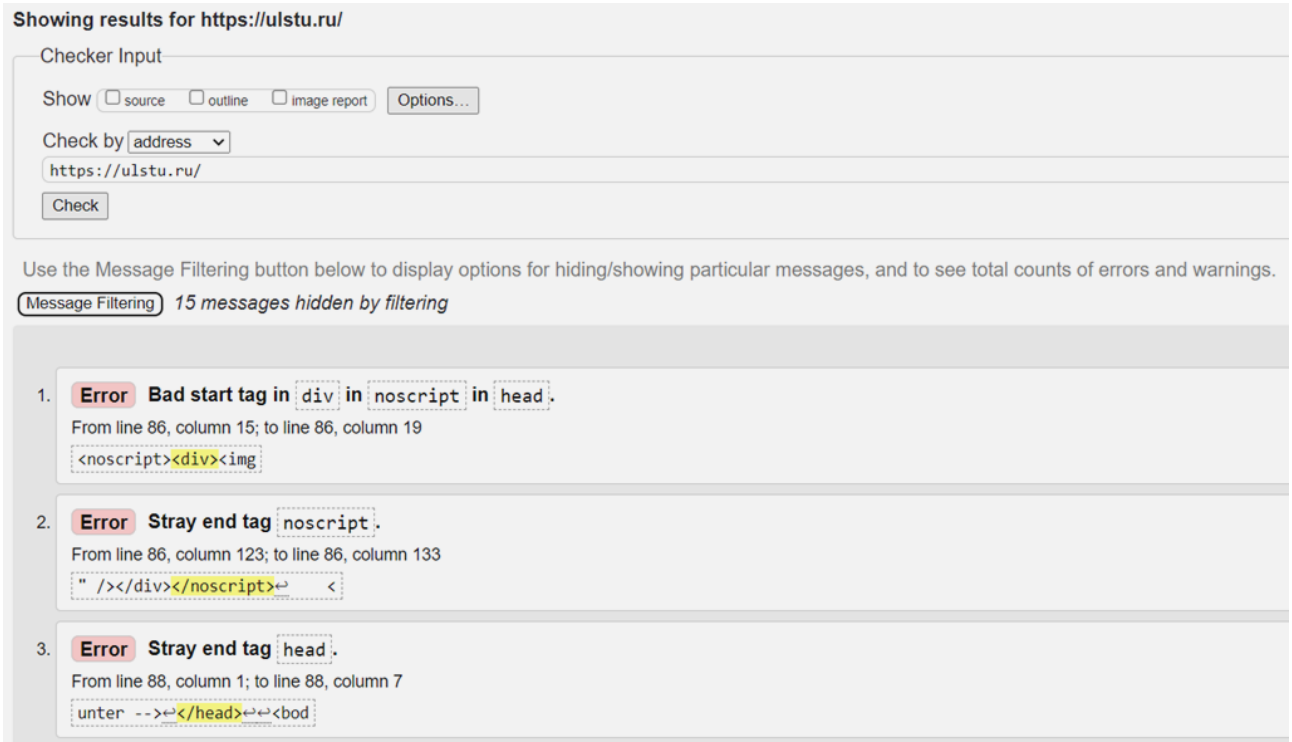


Рис. 2. Отчет валидации html разметки сайта

Динамические методы тестирования позволяют проверять программную систему при ее непосредственном выполнении. Динамические методы делятся на:

– Методы белого ящика. Тестирование на соответствие программного продукта требованиям со знанием внутренней структуры реализации системы (есть в наличии исходный код и технические спецификации). Это вид тестирования позволяет проводить локализацию ошибок, анализ надежности и устойчивости и другие типы проверок, существенно повышая качество системы.

– Методы черного ящика. Тестирование на соответствие программного продукта требованиям без знания внутренней структуры реализации системы.

– Методы серого ящика. Комбинирование методов белого и черного ящиков. При тестировании серого ящика разработчик теста имеет доступ к исходному коду, но при непосредственном выполнении тестов доступ к коду, как правило, не требуется.

4. Виды тестирования, подразделяются на группы:

– по объекту (на что обращен процесс тестирования).
    - Функциональное. Проверяется соответствие требованиям. Оценка производится в соответствии с ожидаемыми и полученными результатами (на основании функциональной спецификации), при условии, что функции отрабатывали на различных значениях.
    - Навигация. Действие кнопки «назад» браузера для сайта, ошибки: 404, 500 и т. д.
    - Инсталляция. Формальное тестирование программы установки (графический интерфейс пользователя, общее удобство пользования, соответствие стандартам), функциональное

тестирование программы установки, тестирование механизма лицензирования и способности противостоять взлому, проверка стабильности работы приложения после установки.

- Нагрузочное. Тесты производительности (количество обрабатываемых запросов в единицу времени), нагрузочные тесты (количество бизнес-пользователей приложения, генерирующих запросы в единицу времени), стресс-тесты (функционирование в условиях экстремальных уровней нагрузки и ограничения ресурсов, а также контроль за скоростью восстановления после стресса), объемное тестирование (увеличение объема хранимых, обрабатываемых данных).
- Ресурсные тесты. Тестирование устойчивости в течение длительного времени.
- Юзабилити. Тестирование удобства пользования приложением определяет, соответствует ли приложение потребностям целевой аудитории и отвечает ли оно требованиям пользователя (сколько шагов и времени нужно пользователям для выполнения их задач в приложении, частота возникновения проблем взаимодействия с пользовательским интерфейсом).
- UI/GUI. Тестирование на соответствие стандартам графических интерфейсов, тестирование с различными разрешениями экрана, тестирование в ограниченных условиях, например, в условиях нехватки памяти, тестирование локализованных версий: точность перевода, проверка длины названий элементов интерфейса, тестирование графического интерфейса пользователя на целевых устройствах.

- Локализация. Проверка правильности перевода согласно тематике данного сайта или программы. Проверка перевода раздела «Помощь» и сопроводительной документации.
- Безопасность. Тестирование безопасности представляет собой ряд услуг, от разработки политики безопасности до тестирования безопасности на уровне приложения, операционной системы и сетевой безопасности. Учитываются три главных критерия: конфиденциальность, целостность, доступность
- Совместимость. Проверяется функционирование на различных операционных системах, платформах разработки, в различных браузерах.
- Конфигурация. Тестируется функционирование программных систем на различных аппаратных платформах и при различных конфигурациях аппаратно-зависимого программного обеспечения (драйвера, взаимодействие с периферийными устройствами).
- Документация. Тестируется на этапе разработки требований к программному продукту после создания функциональных спецификаций. Помогает избежать логических дефектов и ненужных изменений в продукте до начала его фактической разработки. Позволяет улучшить пользовательскую документацию.
- Прототип. Основная цель тестирования прототипа – выявить потенциальные проблемы в приложении, проверить, насколько приложение соответствует потребностям и ожиданиям пользователя, и обнаружить расхождения с требованиями к графическому интерфейсу пользователя. Проверяется:

структура приложения, формы пользовательского интерфейса, прототип бизнес-логики, логические связи между модулями, навигация, графический интерфейс пользователя;

- субъекту, степени автоматизации, признаку позитивности, времени проведения. Данная группа видов тестирования характеризует как выбор объекта тестирования и роль участника команды разработки, так и особенности условий реализации программной системы и жизненного цикла разработки:
    - Субъект тестирования – лицо или группа специалистов, проводящая тестирование (штатные работники, потенциальные заказчики на стороне разработчика, потенциальные пользователи готовой версии продукта с ограничениями).
    - По степени автоматизации: ручное тестирование, автоматизированное тестирование, полуавтоматизированное тестирование.
    - По признаку позитивности. Позитивное тестирование – это тестирование на данных или сценариях, которые соответствуют нормальному (штатному, ожидаемому) поведению системы. Основной целью позитивного тестирования является проверка того, что при помощи системы можно делать то, для чего она создавалась. Негативное тестирование – это тестирование на данных или сценариях, которые соответствуют нештатному поведению тестируемой системы – различные сообщения об ошибках, исключительные ситуации, запредельные состояния. Основной целью негативного тестирования является проверка устойчивости системы к воздействиям различного рода, валидация неверного набора данных, проверка обработки исключительных ситуаций

(как в реализации самих программных алгоритмов, так и в логике бизнес-правил);

- По времени проведения. Альфа-тестирование проводится на ранней стадии разработки продукта, но в некоторых случаях может применяться для законченного продукта в качестве внутреннего приемочного тестирования. Иногда альфа-тестирование выполняется под отладчиком или с использованием окружения, которое помогает быстро выявлять найденные ошибки. Тестирование новой функциональности проверяет корректность реализации новых задач. Тестирование при приемке (Smoke-testing, дымное тестирование) – поверхностная проверка всех модулей приложения на предмет работоспособности и наличия быстронаходимых критических и блокирующих дефектов. Тестирование сборки направлено на определение соответствия, выпущенной версии критериям качества для начала тестирования. Приемочное тестирование – формальный процесс тестирования, который проверяет соответствие системы требованиям и проводится с целью определения, удовлетворяет ли система приемочным критериям, вынесения решения заказчиком или другим уполномоченным лицом о приемке приложения. Регрессионное тестирование – это вид тестирования, направленный на проверку изменений, сделанных в приложении или окружающей среде (починка дефекта, слияние кода, миграция на другую операционную систему, базу данных, веб-сервер или сервер приложения), для подтверждения того факта, что существующая ранее функциональность работает как и

прежде. Регрессионными могут быть как функциональные, так и нефункциональные тесты.

Задания к лабораторной работе

1. Выбрать 2-3 вида тестирования, наиболее характерно подтверждающих качество разработанного проекта.
2. Провести тестирование.
3. Сформировать отчет по тестированию. Указать найденные дефекты, время и периодичность (если была) проведения тестирования.

Контрольные вопросы

1. Какие виды тестирования можно применить на данном этапе разработки?
2. Какие методы тестирования можно применить на данном этапе разработки?
3. При помощи каких инструментов можно автоматизировать тестирование?
4. Как оценить возможность и востребованность автотестирования на проекте?
5. Какая документация участвует и формируется при тестировании?

КУРСОВОЙ ПРОЕКТ

Целью курсового проекта является закрепление и углубление теоретических знаний по дисциплине, получение навыков планирования, построения процесса конструирования, тестирования и их применение для разработки программного продукта.

Объем работы не должен превышать 20–30 страниц, выполненных машинописным способом.

Требования, предъявляемые к курсовой работе:

- самостоятельная реализация и оформление ее в виде веб-сервиса или мобильного приложения;
- четкое разделение интерфейса и логики приложения;
- наличие детализации и оценок по задачам для сопоставления с результатами работы;
- четко и грамотно излагать и правильно оформлять работу в целом;
- отвечать основным правилам оформления курсовых работ.

Содержание курсовой работы определяется характером темы, но, как правило, состоит из введения, нескольких глав и заключения.

Во введении курсовой работы приводится обоснование актуальности темы и краткое описание предметной области. В первой главе приводится постановка задачи, далее идет описание функций системы и описание процесса проектирования, перечень исходных данных, планирование проекта как с точки зрения определения объемов работ, так и с точки зрения организации разработки, выбора и обоснования методологии. Вторая глава посвящена проектированию и разработке. Здесь описывается выбор средств разработки, использование сторонних библиотек, использование системы контроля версий, а также основные главы программы. Далее идет глава, содержащая описания процесса тестирования программного продукта, как на этапе разработки, так и после

него. В заключении следует подвести общие итоги, кратко изложить полученные результаты, оценить фактически затраченное время и ресурсы, дать оценку расхождения по сравнению с первоначальным планом. Список использованных источников должен включать не менее 5 источников, использованных при написании курсовой работы. Приложение курсовой работы включает в себя листинг программного кода.

Защита курсовой работы состоит из краткого изложения студентом основных положений работы, ответов на заданные вопросы. Общий объем курсового проекта должен составлять примерно 20-30 страниц (включая листинг программного кода). Правильно оформленная работа должна включать в себя:

1. Титульный лист.
2. Содержание.
3. Введение (содержит описание состояния проблемы, актуальность, цели и задачи проекта, объекта и предмета).
4. Планирование разработки проекта.
5. Описание проектирования.
6. Описание, обоснование выбора процедур процесса конструирования: методология, стратегия, используемые языки, инструменты.
7. Техника кодирования (использование соглашений об именовании, форматировании, структурировании кода, техники по организации исходного кода: шаблоны, пакеты, модули; обработка исключительных ситуаций).
8. Качество конструирования (тестирование, интеграция).
9. Заключение (включает выводы и рекомендации).
10. Список использованных источников.
11. Приложения (при необходимости).

Титульный лист оформляется в соответствии с требованиями локальных нормативных актов университета. В содержании перечисляются заглавия всех структурных элементов работы с указанием соответствующих страниц.

Варианты заданий на лабораторные работы и курсовой проект:

1. Расписание университета
2. Система управления проектами для студенческого совета
3. Интернет-магазин игрушек
4. Мониторинг веб-сервисов
5. График работы персонала кафе
6. Автомобильное тюнинг-ателье
7. Составление расписания занятий
8. Планирование и ремонт оборудования
9. Оценка работы исполнителей проекта
10. Учет научной активности преподавателей
11. Удаленное управление сервером

ПЕРЕЧЕНЬ КОНТРОЛЬНЫХ ВОПРОСОВ К ЭКЗАМЕНУ

1. Понятие конструирования программных средств.
2. Место конструирования в жизненном цикле программного обеспечения.
3. Методология SWEBOK.
4. Методология FDD.
5. SCRUM.
6. Принципы agile.
7. Методология Rational Unified Process (RUP).
8. Методология Microsoft Solutions Framework (MSF).
9. Стандарты и модели конструирования.
10. Модели жизненного цикла.
11. Классический (каскадный) жизненный цикл. Макетирование. Инкрементная модель. Быстрая разработка приложений. Спиральная модель. Компонентно-ориентированная модель.
12. Анализ требований и определение спецификаций программного обеспечения.
13. Планирование конструирования.
14. Измерения в конструировании.
15. Проектирование в конструировании.
16. Языки конструирования.
17. Кодирование программного обеспечения.
18. Тестирование в конструировании.
19. Тестирование. Классификация методов.
20. Повторное использование.
21. Качество конструирования.
22. Оценка качественных характеристик программных средств.
23. Оценка трудоемкости и стоимости разработки. Методы оценки.
24. Линейный подход. Метод lines of code (LOC).
25. Оценка трудоемкости и стоимости разработки программных средств на основе метода функциональных точек и его разновидностей.

26. Оценка трудоемкости и стоимости разработки программных средств с использованием эмпирических данных.
27. Метод функциональных точек.
28. Методика Use Case Points.
29. Метод конструктивной оценки стоимости (COCOMO II).
30. Интеграция программных средств.

СПИСОК ЛИТЕРАТУРЫ

1. Кознов, Д.В. Введение в программную инженерию [Электронный ресурс] : учебное пособие / Д.В. Кознов. – Электрон. дан. – Москва, 2016. – 306 с. – Режим доступа: https://e.lanbook.com/book/100704. – Загл. с экрана (дата обращения: 10.10.2022).

2. Петрухин, В.А. Методы и средства инженерии программного обеспечения [Электронный ресурс] : учебное пособие / В.А. Петрухин, Е.М. Лаврищева. – Электрон. дан. – Москва, 2016. – 467 с. – Режим доступа: https://e.lanbook.com/book/100645. – Загл. с экрана (дата обращения: 10.10.2022).

3. Конструирование программного обеспечения : учебное пособие / составитель А. А. Романов. – Ульяновск : УлГТУ, 2016. – 126 с.

4. Манифест agile [Электронный ресурс] – Доступен в Интернете: https://agilemanifesto.org/ (дата обращения: 10.10.2022).

Учебное издание

РОМАНОВ Антон Алексеевич

КОНСТРУИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Лабораторный практикум

Редактор Н.А. Евдокимова

Подписано в печать 18.11.2022. Формат 60x84/16.
Усл. печ. л. 2,09. Тираж 35 экз. Заказ 573. ЭИ № 1756.

Ульяновский государственный технический университет,
432027, Ульяновск, Сев. Венец, 32.
ИПК «Венец», УлГТУ, 432027, Ульяновск, Сев. Венец, 32.